EveryMovie

Technical Report

Module: COMP3011 Web Services and Web Data
Student: Ilan Ramirez
Student ID: 201777925
Date: 05 March 2026
GitHub Repository: https://github.com/IlanRV/EveryMovie
Live Deployment: https://everymovie.onrender.com
API Documentation: See API_DOCUMENTATION.pdf in the repository root.

1. Introduction & Project Overview

EveryMovie is a movie discovery web application that consumes The Movie Database (TMDB) API to let users explore films by genre and country of origin. Users can search for movies, browse trending titles, view detailed movie information (overview, director, cast, genres, runtime, and rating), and — once registered — create and manage personal movie lists with full CRUD operations backed by a SQLite database.

The application exposes five JSON API endpoints (/genres/, /discover/, /search/, /api/trending/, /api/lists/) alongside traditional HTML page routes, fulfilling the requirement for a data-driven web API with database integration.

2. Technology Stack Justification

Python & Django 6 — Django's batteries-included philosophy provides a built-in ORM, authentication system, templating engine, and CSRF protection, reducing third-party dependencies. Its mature security defaults handle password hashing (PBKDF2-SHA256 with per-user random salts), session management, and protection against common web vulnerabilities (XSS, CSRF, SQL injection). Django was also introduced in the module's practical sessions, providing a solid foundation to build upon.

SQLite — Requires zero configuration and is sufficient for the application's scope: only user-created lists and list items are stored locally, while all movie data is fetched on demand from TMDB. The data volume is modest. For a production system with many concurrent users, PostgreSQL would be a more appropriate choice due to its superior concurrency handling and connection pooling support.

TMDB API v3 — Provides comprehensive, freely accessible movie metadata including genres, credits, ratings, and poster images. Its /discover/movie endpoint supports flexible filtering by genre, country, and sort criteria, aligning directly with the application's core functionality.

Render + Gunicorn + WhiteNoise — Render offers free-tier hosting with automatic deployments from GitHub. Gunicorn serves as the production WSGI server, and WhiteNoise handles efficient static file serving without requiring a separate CDN or web server.

3. Architecture & Design

System Architecture

The application follows a three-tier architecture: the browser makes HTTP requests to the Django backend, which queries the TMDB API for movie data and a local SQLite database for user lists.

1. Frontend — Django templates with vanilla JavaScript making fetch() calls to internal JSON endpoints.
2. Backend — Django views handle routing, TMDB API consumption, data transformation, and database CRUD.
3. Database — SQLite stores user accounts (via Django's auth_user table) and movie lists (MovieList, MovieListItem).

URL & Endpoint Design

Endpoints follow RESTful conventions where practical. JSON API endpoints use descriptive paths: /genres/, /discover/, /search/, /api/trending/, /api/lists/. CRUD operations for lists use resource-oriented URLs: /lists/create/, /lists/<id>/rename/, /lists/<id>/delete/, /lists/<id>/add/, /lists/item/<id>/remove/. Query parameters are used for filtering (?genre=28&country=US&sort=rating), keeping URLs clean and bookmarkable.

Database Schema

Two models support the user lists feature: MovieList (user FK, name, created_at; unique together on user and name) and MovieListItem (movie_list FK, tmdb_id, title, poster_path, added_at; unique together on movie_list and tmdb_id). Only the TMDB movie ID, title, and poster path are stored locally — all other movie data is fetched from TMDB on demand, avoiding data duplication and keeping the database lightweight.

Genre Pair Detection

A notable design decision was the dynamic genre pair detection algorithm. Rather than hard-coding genre combinations, the /genres/ endpoint samples popular and top-rated movies from TMDB and extracts all two-genre combinations that actually appear. This produces realistic pairs like "Action/Comedy" or "Animation/Science Fiction" that reflect TMDB's current catalogue.

Dynamic Quality Filtering

The /discover/ endpoint implements a dynamic quality filter for rating-based sorts. It first probes TMDB to estimate the total catalogue size for the given genre/country combination, then sets a minimum vote count threshold proportional to the catalogue size. This ensures that results are ordered by genuinely well-rated films rather than obscure titles with a single 10/10 vote.

4. Testing Approach

The application includes 8 automated tests across two test classes, run via Django's built-in test framework (python manage.py test). Tests cover three categories:

- Input validation — verifying that missing query parameters return 400 status codes (e.g. /search/ without q, /discover/ without genre and country).
- Authentication enforcement — confirming that unauthenticated requests to protected endpoints like /api/lists/ return 401.
- End-to-end flows — testing signup, login, list creation, and verifying the created list appears correctly in the /api/lists/ JSON response.

Tests that depend on TMDB being reachable accept 502/503 status codes as valid responses rather than failing when the external service is unavailable.

Limitations: Tests do not mock the TMDB API (requiring network access), frontend JavaScript interactions have no automated coverage, and no load or performance testing was conducted.

5. Challenges

- **Deployment issues —** When deploying initially there were a few issues, this skill was very new to me, I had to find the right page, the correct commands and implement the right things in my files. I found many challenges while deploying but it was a good learning experience as I believe I've learned this skill for the future.

- **Dynamic quality filters —** Sorting movies by rating made it so certain obscure movies with few votes got picked as the highest rating movies. This was solved by dynamically setting the vote count threshold based on the catalogue size, fine tuning for niche country and genre combinations.

- **Genre pair rendering —** Applying gradient backgrounds to genre pairs initially broke the rounded pill shape of chips due to CSS border-image overriding border-radius. The fix was to use background gradients instead.

6. Limitations & Future Work

Current Limitations

- SQLite is not suitable for high-concurrency production use. Under heavy load, write operations could block.
- No caching — every page load triggers fresh TMDB API calls, which adds latency and risks hitting TMDB's rate limits.
- No pagination UI — the frontend fetches only the first page of results for discover and search.
- Session-based auth only — the JSON APIs use session cookies rather than token-based authentication (e.g. JWT), making them less suitable for third-party consumers.

Potential Improvements

- PostgreSQL for production database with proper connection pooling.
- Redis caching for TMDB responses to reduce latency and API call volume.
- Infinite scroll / pagination in the frontend for discover and search results.
- Token-based authentication (JWT or API keys) to support external API consumers.
- User ratings and reviews stored locally to complement TMDB data.
- Recommendation engine using collaborative filtering based on users' saved lists.

7. Generative AI Declaration & Analysis

Tools Used

Tool	Purpose
GitHub Copilot (Claude model)	Code generation, debugging, architecture, deployment, docs, CSS

How AI Was Used

GitHub Copilot was used as an integrated development partner throughout the project. I discussed system architecture with Copilot to structure the Django project, design the URL schema, and decide which data to store locally vs. fetch from TMDB on demand. Copilot assisted in writing Django views, models, templates, and JavaScript — for each feature I described the desired behaviour, reviewed the generated code, and made manual adjustments where needed. When issues arose (e.g. genre pair gradients breaking pill shapes, or mobile layout instability), I described the problem and evaluated proposed solutions. Copilot also guided the Render deployment configuration and assisted in drafting documentation, which I then reviewed and personalised.

Analysis & Reflection

AI was an integral part of this coursework. Due to time constraints, it was necessary to reduce

the time spent on low-level coding, and Generative AI was effective at taking my ideas and translating them into working implementations. I felt that I was the imagination, and AI was the toolbox through which I could realize all my ideas.

Working with AI allowed me to see the bigger picture. By stepping back from the code and focusing on architecture and structure, I found a different approach compared to what I usually adopt for coursework, and it encouraged me to pursue innovative solutions as my energy was not wasted on repetitive tasks that might not have the biggest impact.

Although it is useful, Generative AI is not perfect. On several occasions, I found that the AI was making decisions based on its collective training data rather than following my specific vision. Generic solutions were produced which I had to identify and modify. I believe that AI overstepping design decisions at times prevents creativity and promotes conformity, something which I actively tried to avoid throughout the project.

Finally, if AI had not been available as a tool for this coursework, I would not have been able to go to the lengths I did. I would have focused solely on API implementation in its most basic form and would not have attempted to implement features in innovative ways.

Conversation Logs

Selected conversation logs demonstrating AI usage are included as Appendix A (see CONVERSATION_LOGS.pdf in the repository root: https://github.com/IlanRV/EveryMovie/blob/main/CONVERSATION_LOGS.pdf).

References

1. The Movie Database (TMDB) API v3 — https://developer.themoviedb.org/docs

2. Django Documentation (v6.0) — https://docs.djangoproject.com/en/6.0/

3. WhiteNoise Documentation — http://whitenoise.evans.io/

4. Render Deployment Documentation — https://docs.render.com

5. OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html